



ЛЕКЦИЯ 13

Основы нейронных сетей и диагностика процесса обучения

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. · Время изучения: ≈ 75 минут



О чём L13 и зачем она в курсе по безопасности систем

Нейронная сеть – гибкий класс моделей, способный приближать сложные зависимости. Но эта лекция делает акцент не на разнообразии архитектур, а на **понимании процесса обучения и диагностике ошибок**. Вы разберёте, как устроен базовый нейросетевой классификатор, почему он переобучается, как читать кривые обучения и какие признаки указывают на нестабильное обучение.

КЛЮЧЕВОЙ ТЕЗИС

Нейронная сеть – не «чёрный ящик, который сам всё выучит». Качество и надёжность зависят от данных, функции потерь, оптимизации, регуляризации и диагностики обучения. На табличных ИБ-данных сеть, как правило, не превосходит хорошо настроенный бустинг; её главная ценность – в работе с неструктурированными данными.

Дальше: устройство сети → обучение → регуляризация → диагностика кривых → когда сеть вообще оправдана.



Учебные результаты

Вы научитесь:

- объяснять базовую структуру сети: слой, веса, смещение, активация, выход;
- понимать роль функции потерь и оптимизатора, идею градиентного обучения;
- читать learning curves для train и validation;
- различать недообучение, переобучение и нестабильное обучение;
- понимать назначение dropout, weight decay, early stopping и batch normalization;
- объяснять связь качества данных и надёжности нейросетевой модели;
- понимать риски запоминания данных и чрезмерной уверенности softmax;
- аргументированно выбирать между MLP, бустингом и линейным baseline.



Что нужно знать до начала

4 / 65

Часть 1 · Введение

Эта лекция опирается на материал блока 1 и L10

Курс строит знания спирально. К L13 вы уже должны владеть классическим ML и базовой дисциплиной эксперимента. Ниже – конкретные опорные лекции и что именно из них понадобится.

Лекция	Тема	Что понадобится в L13
L06	Линейные модели	score $\langle w, x \rangle + b$, sigmoid $\sigma(z)$, softmax, log-loss – теперь как основа нейросети
L07	Регуляризация	идея регуляризации и weight decay; калибровка и порог – пригодятся в диагностике
L08	Деревья и бустинг	градиентный бустинг – главный конкурент MLP на табличных ИБ-данных
L10	Pipeline	воспроизводимый конвейер и защита от утечки данных при обучении

Не имеет значения предыдущий опыт работы с нейросетевыми библиотеками – строим этот материал с нуля.



План лекции

5 / 65

Часть 1 · Введение

7 частей · ~ 65 слайдов · ~ 75 минут самостоятельного изучения

1	Введение	слайды 1-5	~7 мин
2	Мотивация: зачем сеть в ИБ	слайды 6-10	~7 мин
3	Основное содержание (блоки А-Е)	слайды 11-53	~50 мин
4	Связки с практикой ИБ	слайды 54-60	~9 мин
5	Итоги	слайды 61-62	~3 мин
6	Источники для углубления	слайды 63-64	~2 мин
7	Глоссарий	слайд 65	~2 мин



Зачем вообще нейросеть в ИБ

6 / 65

Часть 2 · Мотивация

Сдержанный ответ: не везде – но там, где она нужна, заменить её ничем

На табличных задачах ИБ – NetFlow-признаки, статические признаки файлов, агрегаты транзакций – у вас уже есть сильный инструмент: **градиентный бустинг из L08**. Он быстро обучается, устойчив и почти не требует тонкой настройки. Нейросеть здесь, как правило, не даёт выигрыша и стоит дороже по усилиям.

Настоящая ценность сети – там, где признаки нельзя задать руками: **изображения, тексты, логи как последовательности**. Сеть учит представление данных сама. Именно поэтому L13 – фундамент для следующих лекций блока 2 и блока 3.

L14

Изображения

CNN и переносимость качества

L15

Тексты и логи

от TF-IDF к эмбедингам

L16

LLM в ИБ

возможности, ограничения, риски



Проблема: когда ручные признаки не работают

7 / 65

Часть 2 · Мотивация

Граница применимости классических моделей

Линейные модели и деревья работают с **готовым вектором признаков** $x \in \mathbb{R}^d$. Кто-то должен этот вектор построить: посчитать длительность сессии, число пакетов, энтропию строки, частоты байтов. На табличных данных это естественно — признаки уже заданы инженером.

Но для сырого трафика-payload, исполняемого файла как байтовой последовательности, текста фишингового письма или картинки CAPTCHA **хорошего ручного набора признаков может не существовать**. Здесь нужна модель, которая выстраивает представление сама из сырых данных — это и есть нейронная сеть.

В ДВУХ СЛОВАХ

Нейросеть нужна не «потому что мощнее», а потому что снимает зависимость от ручного конструирования признаков там, где его выполнить трудно или невозможно.



ИБ-кейс: соблазн «глубже – значит лучше»

8 / 65

Часть 2 · Мотивация

Реальный антипаттерн при детектировании сетевых атак

ИБ-КЕЙС · ДЕТЕКТ СЕТЕВЫХ АТАК

Команда SOC строит детектор на табличных NetFlow-признаках. **Бустинг** за полчаса даёт ROC-AUC ≈ 0.98 .

Аналитик решает «нейросеть точно сильнее» и неделю настраивает MLP: слои, dropout, learning rate, число эпох.

Итог – те же ≈ 0.98 , но в 20 раз больше потраченного времени и менее объяснимая модель.

Что пошло не так:

- архитектуру выбрали по интуиции «мощнее = лучше», а не по типу данных;
- не сделали честное сравнение с baseline по метрике И по стоимости настройки;
- получили модель, которую труднее объяснить аудитору и сложнее поддерживать.



Типовые провалы инженера в реальных ML-системах ИБ

Без навыка диагностики обучения сеть превращается в источник тихих ошибок. Вот что происходит на практике, когда инженер смотрит только на итоговую метрику:

ЧАСТАЯ ОШИБКА · ОЦЕНКА ТОЛЬКО ПО ACCURACY

«Accuracy 0.99 – модель отличная».

На несбалансированных ИБ-данных (атак – доли процента) accuracy 0.99 может означать модель, которая просто всё считает «нормой».

«Loss падает – значит, учится хорошо».

Падает train loss, а validation loss уже растёт – это переобучение, незаметное без взгляда на обе кривые.

ПОЧЕМУ ЭТО ОПАСНО В ИБ

Модель, прошедшая «по accuracy», в проде пропускает реальные атаки или заваливает аналитика ложными срабатываниями. Цена ошибки – не точка на графике, а инцидент. Диагностика обучения – это техника безопасности, а не формальность.



Один и тот же датасет

CIC-IDS-2018 · ЧТО ЭТО

Публичный датасет сетевого трафика. ≈ 16 млн flow-записей за 10 дней, 80 признаков на запись, 14 типов атак плюс класс Benign. Используется как сквозной пример курса: одна и та же задача обнаружения подозрительного трафика решается всё более продвинутыми инструментами – от линейных моделей до нейросетей.

Роль в L13:

это **табличная задача**. Мы обучим на ней MLP на тех же признаках, на которых в L08 обучался бустинг, и проведём честное сравнение – по метрикам и по затратам на настройку. Это центральный аргумент линии «baseline-дисциплина»: **сеть нужно сравнивать с бустингом, а не брать по умолчанию.**

Подробное сравнение MLP и бустинга на CIC-IDS-2018 – в Части 4 (слайд 54).



Из чего состоит сеть

Базовая анатомия нейросетевого классификатора

В этом блоке:

- нейрон как вычислительный элемент: $z = \langle w, x \rangle + b$ и активация;
- слой и полносвязная сеть (MLP);
- активации ReLU, sigmoid, softmax – зачем нужна нелинейность;
- forward pass, функция потерь и почему «глубже» – не всегда «лучше».



Тот же score, что в линейной модели L06 – плюс нелинейность

Один нейрон делает ровно то, что вы видели в линейной модели: берёт вход x , считает взвешенную сумму со смещением, а затем пропускает результат через **нелинейную активацию** f . Именно активация отличает нейрон от линейного классификатора.

ФОРМУЛА · ВЫЧИСЛЕНИЕ ОДНОГО НЕЙРОНА

$$z = \langle w, x \rangle + b \qquad a = f(z)$$

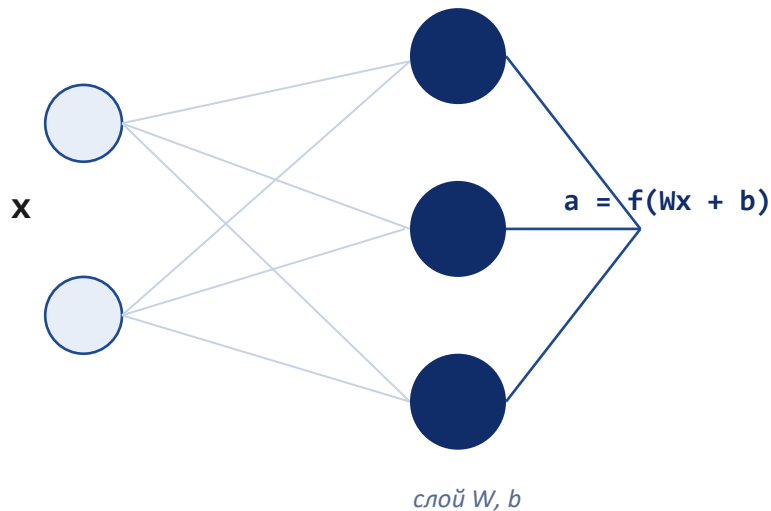
$x \in \mathbb{R}^d$ – вход (признаки объекта); w – веса нейрона; b – смещение; z – предактивация (логит); f – активация; a – выход нейрона.

Связь с L06: при $f(z) = z$ нейрон вырождается в линейную модель, а при $f(z) = \sigma(z)$ – в логистическую регрессию. Сеть получает выразительность не из одного нейрона, а из их слоёв и нелинейностей между ними.



Слой – это набор нейронов, считающих свои выходы параллельно

Слой – это несколько нейронов, которые получают один и тот же вход и считают свои выходы независимо. Их веса удобно собрать в **матрицу W** , а смещения – в вектор b . Тогда весь слой считается одной операцией: $z = Wx + b$, $a = f(z)$. Число нейронов в слое называют его **шириной**.



Несколько слоёв подряд, где выход одного – вход следующего, образуют сеть. Промежуточные слои называют скрытыми.



Multilayer Perceptron – базовая архитектура этой лекции

ОПРЕДЕЛЕНИЕ · MLP

Многослойный персептрон (MLP) – нейронная сеть из последовательных полносвязных слоёв, где каждый нейрон слоя связан со всеми выходами предыдущего. Вход $x \in \mathbb{R}^d$ проходит через скрытые слои с нелинейными активациями и завершается выходным слоем, дающим prediction.

Три роли слоёв:

Входной

принимает вектор признаков $x \in \mathbb{R}^d$ – по нейрону на признак

Скрытые

строят внутреннее представление; их число и ширина задают ёмкость модели

Выходной

даёт ответ: 1 нейрон + sigmoid для бинарной задачи, K нейронов + softmax для K классов

На этом уровне детали не важны: запомните, что MLP – это «стопка» слоёв, превращающая x в ответ.



Зачем нужна нелинейность и где какая активация уместна

Без активаций стопка линейных слоёв схлопывается в одну линейную модель – какой бы глубокой ни была сеть.

Нелинейная активация между слоями – это то, что даёт сети способность приближать сложные зависимости.

ReLU

$$f(z) = \max(0, z)$$

Скрытые слои. Дёшево, не «затухает», стандартный выбор по умолчанию.

sigmoid

$$\sigma(z) = 1/(1+e^{-z})$$

Выход бинарной задачи: сжимает score в (0, 1). В скрытых слоях почти не используют.

softmax

над вектором логитов

Выход для K классов: даёт распределение, сумма = 1. Продолжение из L06.

Правило по умолчанию: ReLU в скрытых слоях, sigmoid или softmax – только на выходе, в зависимости от числа классов.



Как сеть превращает логиты в вероятности по классам

Выходной слой сети для K классов даёт K чисел – логитов $z_1 \dots z_K$. Сами по себе они не вероятности: могут быть любого знака и не дают в сумме единицу. **Softmax** превращает их в распределение – продолжение sigmoid из L06 на K классов.

ФОРМУЛА · SOFTMAX ДЛЯ КЛАССА k

$$\text{softmax}(z)_k = \exp(z_k) / \sum_j \exp(z_j)$$

Результат – вектор из K чисел в (0, 1), сумма которых равна 1. Каждое число интерпретируют как «оценку уверенности» сети в соответствующем классе.

ВАЖНАЯ ОГОВОРКА

Сумма = 1 ещё не делает эти числа надёжными вероятностями. Почему высокий softmax-score может обманывать – разберём отдельно на слайде 45.



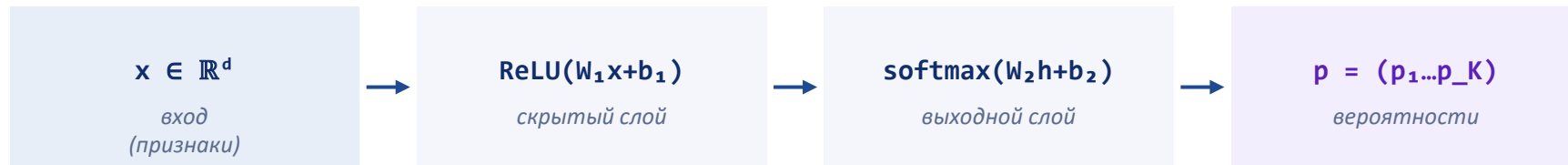
Forward pass: путь от входа к ответу

17 / 65

Часть 3 · Блок А

Прямой проход – это просто последовательное применение слоёв

Forward pass (прямой проход) – это вычисление ответа сети: вход последовательно проходит все слои, на каждом считается $z = Wx + b$ и применяется активация. Выход последнего слоя – prediction.



ЧИСЛОВОЙ ПРИМЕР · 3 КЛАССА

Логиты выходного слоя: $z = (2.0, 1.0, 0.1)$.

После softmax: $p \approx (0.66, 0.24, 0.10)$. Сумма = 1. Сеть «уверена» в классе 1 на 66%.

Prediction = argmax = класс 1. Весь forward pass – это арифметика: умножения, сложения и активации, никакого «магического вывода».



Чем измеряют ошибку сети

Forward pass даёт предсказанное распределение p . Чтобы обучать сеть, нужна мера её ошибки – **функция потерь $L(w)$** . Для классификации это кросс-энтропия: она штрафует за низкую вероятность, назначенную правильному классу. Это прямое продолжение log-loss из L06.

ФОРМУЛА · КРОСС-ЭНТРОПИЯ (на N объектах)

$$L(w) = - (1/N) \sum_i \log p(y_i | x_i)$$

$p(y_i | x_i)$ – вероятность, которую сеть назначила истинному классу объекта i ; N – число объектов. Чем ниже эта вероятность, тем больше $-\log$ и тем сильнее штраф.

Обучение = минимизация $L(w)$: ищем веса, при которых сеть в среднем назначает высокую вероятность правильным ответам. Как именно ищем – в блоке В.



Ошибка: «глубже – значит лучше»

19 / 65

Часть 3 · Блок А

Глубина без необходимости – источник проблем, а не качества

ЧАСТАЯ ОШИБКА · ГОНКА ЗА ГЛУБИНОЙ

«Добавлю ещё слоёв и нейронов – модель станет точнее».

Студенты наращивают глубину и ширину, считая, что это безусловно повышает качество. На практике сверх некоторого предела это ухудшает результат, особенно на небольших табличных ИБ-данных.

ПОЧЕМУ ЭТО НЕВЕРНО

Переобучение. Лишняя ёмкость позволяет сети запоминать шум и артефакты обучающей выборки вместо закономерностей.

Нестабильность обучения. Чем глубже сеть, тем труднее её обучать: градиенты затухают или взрываются, результат зависит от инициализации.

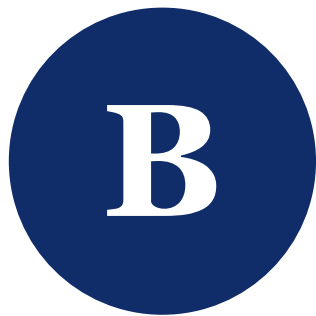
Цена без выгоды. Больше параметров – дольше обучение, больше данных нужно, сложнее объяснить модель. А прироста качества может не быть вовсе.



Базовая анатомия сети – фундамент для всего дальнейшего

Что мы только что разобрали:

- ✓ Нейрон считает $z = \langle w, x \rangle + b$ и пропускает через нелинейную активацию; именно нелинейность отличает его от линейной модели.
- ✓ Слой – набор нейронов (W, b); MLP – стопка полносвязных слоёв: вход $\rightarrow$ скрытые слои $\rightarrow$ выходной слой с prediction.
- ✓ ReLU – в скрытых слоях; sigmoid или softmax – на выходе. Softmax даёт распределение по классам (сумма = 1), но ещё не надёжную вероятность.
- ✓ Forward pass – это арифметика по слоям; функция потерь (кросс-энтропия) измеряет ошибку, продолжая log-loss из L06.
- ✓ «Глубже» – не значит «лучше»: лишняя ёмкость ведёт к переобучению, нестабильности и затратам без прироста качества.



Как сеть обучается

От функции потерь к подбору весов

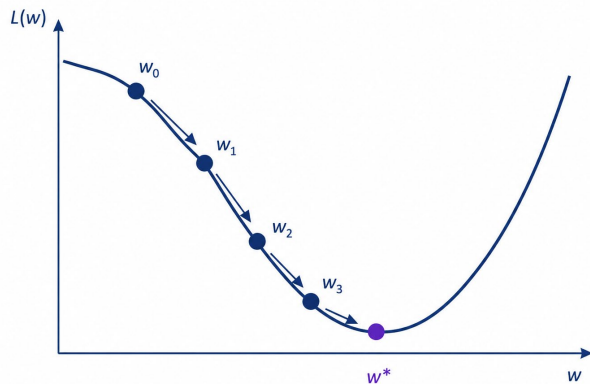
В этом блоке:

- идея градиентного обучения: спуск по поверхности потерь;
- backpropagation как способ вычислить направление (chain rule, без вывода);
- оптимизаторы SGD и Adam на уровне идеи;
- epoch, batch, learning rate – что это и что настраивать.



Обучение – это спуск по поверхности функции потерь

Функция потерь $L(w)$ задаёт «ландшафт»: каждой комбинации весов соответствует своя ошибка. Обучение – это поиск низины этого ландшафта. **Градиент** показывает направление самого быстрого роста $L(w)$; значит, чтобы уменьшить ошибку, нужно делать шаг в противоположную сторону – по антиградиенту.



точки — шаги градиентного спуска к минимуму

Размер шага задаёт **learning rate**. Слишком большой – спуск «перепрыгивает» низину и расходится; слишком малый – обучение идёт мучительно долго.



Backpropagation за один слайд

23 / 65

Часть 3 · Блок В

Как сеть вычисляет градиент по всем весам сразу

Чтобы сделать шаг спуска, нужен градиент $L(w)$ по каждому весу. В сети из многих слоёв это считается **обратным проходом (backpropagation)**: ошибка с выхода распространяется назад по слоям, и на каждом слое применяется цепное правило (chain rule).

ИДЕЯ · ЦЕПНОЕ ПРАВИЛО (chain rule)

$$\partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial w$$

Читается справа налево: вклад веса w в ошибку — это произведение локальных «чувствительностей» вдоль пути от веса к выходу. Backpropagation просто организует это вычисление эффективно для всей сети сразу.

ГРАНИЦА ТЕМЫ

Полный математический вывод backpropagation в этом курсе не разбирается. Достаточно понимать идею: это автоматический и эффективный способ получить направление изменения весов.



Кто именно делает шаги по антиградиенту

Backpropagation даёт направление, а **оптимизатор** решает, как именно обновлять веса на каждом шаге. Два базовых варианта, которые вы встретите чаще всего:

SGD

Стохастический градиентный спуск

Делает шаг по антиградиенту на мини-батче. Прост и надёжен, но чувствителен к выбору learning rate и может сходиться медленно.

Adam

Адаптивный момент

Подстраивает шаг под каждый вес, накапливая историю градиентов. Часто сходится быстрее «из коробки» — разумный выбор по умолчанию.

На уровне этого курса: начните с Adam и стандартного learning rate. Тонкая настройка расписаний шага (lr schedules) выходит за рамки лекции.



Epoch, batch, learning rate

25 / 65

Часть 3 · Блок В

Три понятия, которыми описывают процесс обучения

ОПРЕДЕЛЕНИЕ · BATCH

Batch – мини-выборка объектов, по которой считается один шаг обновления весов. Весь датасет за раз обычно не используют – это дорого и менее устойчиво.

ОПРЕДЕЛЕНИЕ · EPOCH

Epoch – один полный проход по всем обучающим данным. Обучение длится несколько эпох; число эпох контролируют по validation loss (см. early stopping).

ОПРЕДЕЛЕНИЕ · LEARNING RATE

Learning rate – размер шага спуска. Ключевой гиперпараметр: слишком большой – расходимость, слишком малый – медленное обучение.



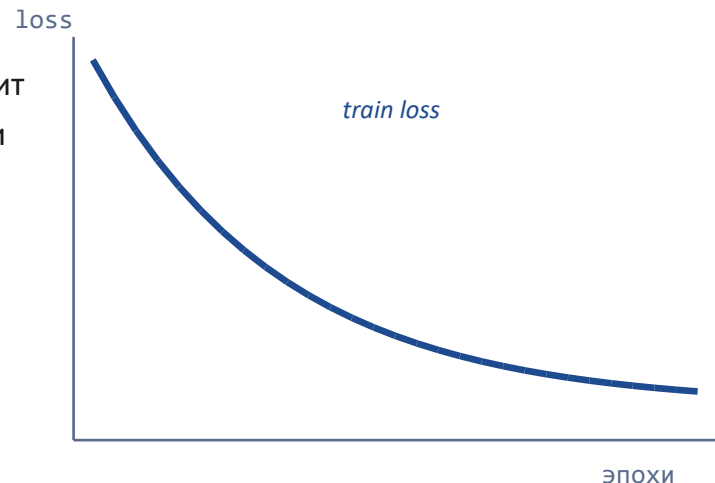
Что значит «обучение пошло»

26 / 65

Часть 3 · Блок В

Нормальная динамика: train loss убывает по эпохам

Признак того, что сеть учится, – **снижение train loss от эпохи к эпохе**. В начале потеря падает быстро, затем замедляется и выходит на плато: сеть выбрала почти всё, что могла извлечь из данных при текущей архитектуре.



ВНИМАНИЕ

Падение train loss само по себе ещё ничего не гарантирует. Хорошо сеть обучилась или просто запоминает обучающую выборку – покажет только validation loss. Об этом – весь блок D.



Тот же датасет, что в L08 для бустинга – теперь нейросеть

ИБ-КЕЙС · НАСТРОЙКА ОБУЧЕНИЯ

Задача: бинарная классификация flow-записей (Benign / атака) на тех же признаках, что использовал бустинг в L08.

Конфигурация: MLP с двумя скрытыми слоями (ReLU), выход sigmoid, оптимизатор Adam, batch = 256, до 40 эпох, фиксированный seed.

Как падала потеря по эпохам:

Эпоха	train loss	val loss	Что происходит
1	0.62	0.60	быстрый старт: сеть ловит грубые закономерности
10	0.18	0.19	обе потери снижаются согласованно – здоровое обучение
25	0.09	0.12	train ниже validation: разрыв пока небольшой
40	0.04	0.15	train падает, validation растёт – начинается переобучение

Вывод напрашивается заранее: дольше – не значит лучше. Где остановиться – решает early stopping (блок С).



Ошибка: «крутить эпохи без контроля»

28 / 65

Часть 3 · Блок В

Чем больше учим – тем лучше? Не всегда

ЧАСТАЯ ОШИБКА · ПОГОНЯ ЗА ЭПОХАМИ

«Train loss ещё падает – продолжу учить, станет точнее».

Инженер увеличивает число эпох, ориентируясь только на train loss, и радуется, что ошибка на обучении почти ноль. На самом деле сеть давно перестала обобщать и начала запоминать выборку.

ПОЧЕМУ ЭТО НЕВЕРНО

Train loss всегда можно догнать почти до нуля – достаточно дать сети достаточно эпох и параметров. Но это говорит лишь о запоминании, а не о качестве на новых данных. Единственный честный индикатор – **validation loss**: как только он перестаёт снижаться и начинает расти, обучение пора останавливать, сколько бы ни падал train loss.

Инструмент, который автоматизирует «где остановиться», – early stopping. Разберём в блоке С (слайд 33).



Как сеть превращает данные в подобранные веса

Что мы только что разобрали:

- ✓ Обучение – это спуск по поверхности потерь $L(w)$: шаг делается по антиградиенту, его размер задаёт learning rate.
- ✓ Backpropagation вычисляет градиент по всем весам через цепное правило; полный вывод знать не требуется – важна идея.
- ✓ Оптимизатор (SGD, Adam) решает, как обновлять веса; разумный выбор по умолчанию – Adam со стандартным learning rate.
- ✓ Epoch – проход по всем данным, batch – мини-выборка для одного шага; число эпох контролируют по validation loss.
- ✓ Падающий train loss сам по себе ничего не гарантирует: дольше учить – не значит лучше. Честный индикатор – validation loss.



Регуляризация и стабильность

Как удержать сеть от переобучения и нестабильного обучения

В этом блоке:

- dropout – случайное отключение нейронов при обучении;
- weight decay – регуляризация весов (концепция из L07);
- early stopping – остановка по validation loss;
- batch normalization, инициализация и фиксация seed.



Случайное отключение нейронов как защита от переобучения

ОПРЕДЕЛЕНИЕ · DROPOUT

Dropout – приём регуляризации: на каждом шаге обучения часть нейронов слоя случайно «отключается» (их выход обнуляется). На инференсе работают все нейроны. Доля отключаемых – гиперпараметр (обычно 0.2-0.5).

Зачем это работает: сеть не может полагаться на отдельные нейроны-«звёзды» и вынуждена выучивать избыточные, более устойчивые представления. Это мешает ей запоминать конкретные обучающие примеры и снижает переобучение.



активные нейроны – синие, отключённые на этом шаге – серые пунктирные



Та же регуляризация весов, что в L07 – теперь для сети

Концепция вам знакома из L07: к функции потерь добавляется штраф за большие веса. Сеть платит не только за ошибки на данных, но и за «раздувание» весов, поэтому предпочитает более простые, гладкие решения.

ФОРМУЛА · ПОТЕРЯ СО ШТРАФОМ (L2)

$$L_{\text{total}}(\mathbf{w}) = L(\mathbf{w}) + \lambda \cdot ||\mathbf{w}||^2$$

λ – сила регуляризации: больше λ – сильнее «прижимаются» веса. $||\mathbf{w}||^2$ – сумма квадратов весов. В нейросетях этот штраф и называют weight decay; в Adam он задаётся одним параметром.

СВЯЗЬ С КУРСОМ

Это прямое продолжение линии регуляризации из L07. Идея одна и та же – управлять сложностью модели через штраф; меняется лишь объект, к которому она применяется.

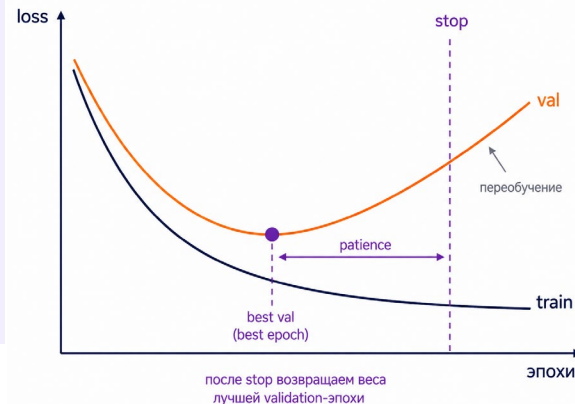


Останавливаем обучение там, где validation loss минимален

ОПРЕДЕЛЕНИЕ · EARLY STOPPING

Early stopping – правило ранней остановки: обучение прекращают, когда validation loss перестаёт снижаться (с небольшим запасом терпения). Веса возвращают к лучшей по validation эпохе.

Двойной эффект: early stopping и предотвращает переобучение (не даёт сети «уйти» в запоминание), и экономит время – не тратим эпохи, которые только ухудшают обобщение. Это самый дешёвый и почти бесплатный вид регуляризации.





Стабилизация обучения через нормализацию выходов слоёв

По мере обучения распределение значений внутри сети «плавает» от слоя к слою и от шага к шагу. Это затрудняет обучение: каждый слой вынужден постоянно подстраиваться под меняющийся вход. **Batch normalization** нормализует выходы слоя по статистике мини-батча – приводит их к стабильному масштабу.

Что это даёт на практике:

- обучение идёт устойчивее и часто быстрее – можно использовать больший learning rate;
- сеть менее чувствительна к неудачной инициализации весов;
- даёт лёгкий побочный регуляризующий эффект за счёт шума мини-батча.

НА УРОВНЕ ИДЕИ

Запомните назначение, а не формулу: batch normalization – это «выравниватель масштаба» внутри сети, который делает обучение более предсказуемым. Точные выкладки в этом курсе не разбираются.



Как выглядит обучение, которое «не пошло»

Веса сети инициализируют случайно. **Неудачная инициализация** (или слишком большой learning rate) приводит к нестабильному обучению: его видно по форме кривой потерь ещё до всякой оценки качества.



СИМПТОМ → ДЕЙСТВИЕ

Скачки и расходимость – сигнал уменьшить learning rate, проверить инициализацию и масштаб признаков (нормализацию входа), а не «учить ещё дольше».



Случайность обучения – враг честного сравнения моделей

ЧАСТАЯ ОШИБКА · ПЛАВАЮЩИЕ РЕЗУЛЬТАТЫ

«Запустил дважды – получил разные числа, наверное, это нормально».

Инициализация весов, перемешивание батчей и dropout используют случайность. Без фиксации seed каждый запуск даёт свой результат, и сравнивать модели становится невозможно.

ПОЧЕМУ ЭТО ОПАСНО

Нельзя отличить реальное улучшение от случайной флуктуации. «MLP обошёл бустинг на 0.3%» может быть просто везением одного запуска. Без фиксированного seed вывод о превосходстве модели не имеет силы.

Связь с курсом: это та же дисциплина воспроизводимого эксперимента, что в L05 и L10 – фиксируем seed, версии, разбиение данных, чтобы результат можно было повторить и честно сравнить.



Инструменты против переобучения и нестабильности

Что мы только что разобрали:

- ✓ Dropout случайно отключает нейроны при обучении, заставляя сеть выучивать устойчивые избыточные представления.
- ✓ Weight decay – штраф за большие веса (L2), прямое продолжение регуляризации из L07; в Adam задаётся одним параметром.
- ✓ Early stopping останавливает обучение в минимуме validation loss: предотвращает переобучение и экономит время.
- ✓ Batch normalization стабилизирует масштаб внутри сети – обучение идёт устойчивее и менее чувствительно к инициализации.
- ✓ Нестабильность видна по форме кривой (скачки, расходимость); фиксируйте seed – иначе сравнивать модели невозможно.



Диагностика обучения

Главный практический навык лекции: чтение кривых обучения

В этом блоке:

- train loss и validation loss – что показывает каждая;
- как читать learning curves и что значит расхождение кривых;
- три режима: недообучение, переобучение, нестабильное обучение;
- ловушка softmax-уверенности и чек-лист минимальной диагностики.



Две кривые, по которым видно всё о процессе обучения

ОПРЕДЕЛЕНИЕ · ДВЕ КРИВЫЕ ПОТЕРЬ

Train loss – ошибка на обучающей выборке: показывает, насколько сеть подогналась под данные, которые видела.

Validation loss – ошибка на отложенной выборке, которую сеть не использовала для обновления весов: показывает, как модель обобщает на новые данные.

Смотреть нужно обе. Train loss отвечает на вопрос «сеть вообще учится?», validation loss – на главный вопрос «будет ли она работать на данных, которых не видела?». Качество модели определяет именно вторая кривая.

КЛЮЧЕВАЯ ИНТУИЦИЯ

Разрыв между кривыми (val выше train) – это цена обобщения. Небольшой разрыв нормален; растущий разрыв – сигнал переобучения. Об этом – следующие слайды.



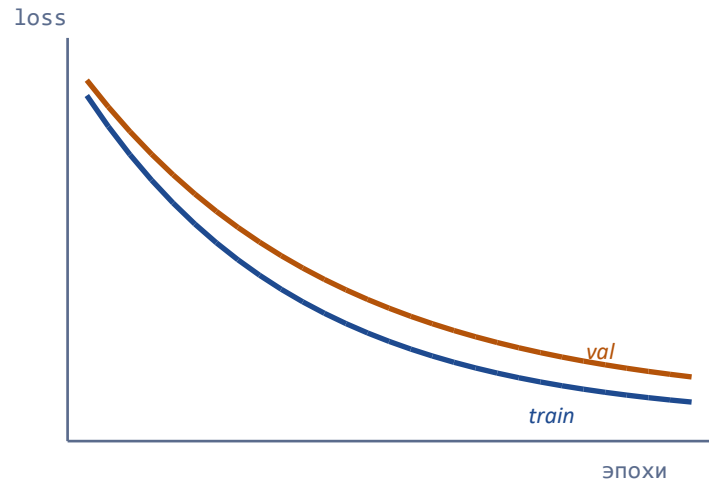
Как читать learning curves

40 / 65

Часть 3 · Блок D

Эталон: как выглядит здоровое обучение

На графике по горизонтали – эпохи, по вертикали – потеря. Рисуют обе кривые на одних осях. **Здоровое обучение:** обе потери снижаются, идут близко друг к другу и выходят на плато. Небольшой устойчивый разрыв между ними – норма.



На что смотреть: (1) снижаются ли обе кривые; (2) насколько велик разрыв между ними; (3) растёт ли val loss после некоторой эпохи. Эти три вопроса задают всю диагностику.



Режим 1: недообучение

41 / 65

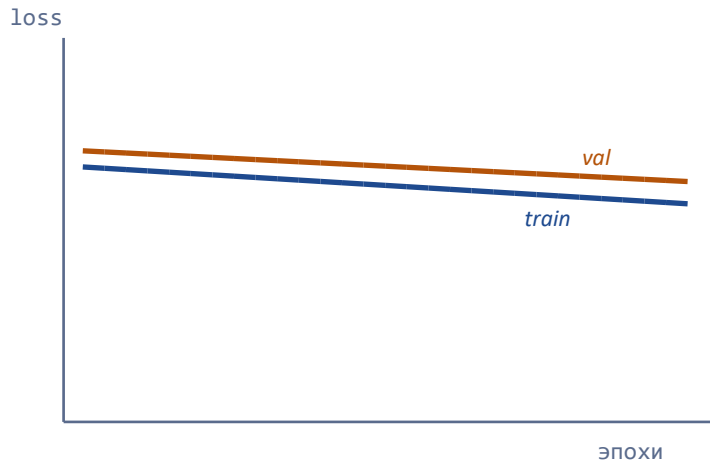
Часть 3 · Блок D

Обе потери высоки и почти не снижаются

Симптом: и train, и validation loss остаются высокими и слабо падают. Сети не хватает способности уловить закономерность – она недообучена.

Что делать:

- увеличить ёмкость: больше слоёв или нейронов;
- учить дольше, проверить learning rate;
- ослабить регуляризацию, если она слишком сильная;
- добавить или улучшить признаки.



ПРИЗНАК

Кривые идут высоко и почти параллельно оси — сеть «не зацепилась» за данные.



Режим 2: переобучение

42 / 65

Часть 3 · Блок D

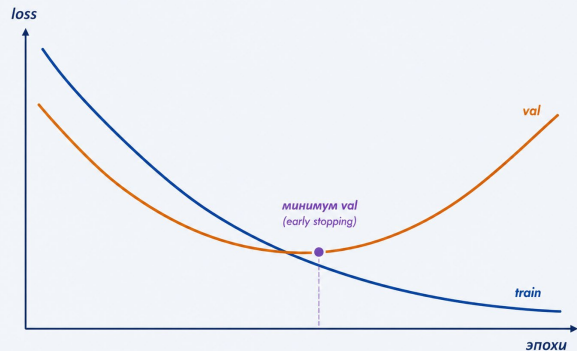
Train падает, validation разворачивается вверх

Симптом: train loss продолжает падать (вплоть до около нуля), а validation loss сначала снижается, затем разворачивается и растёт. Разрыв между кривыми увеличивается – сеть запоминает обучающую выборку.

Что делать:

- early stopping в точке минимума val loss;
- усилить регуляризацию: dropout, weight decay;
- уменьшить ёмкость сети; добавить данные.

Переобучение



ПРИЗНАК

«Ножницы»: train вниз, val вверх. Растущий разрыв – главный маркер переобучения.



Режим 3: нестабильное обучение

43 / 65

Часть 3 · Блок D

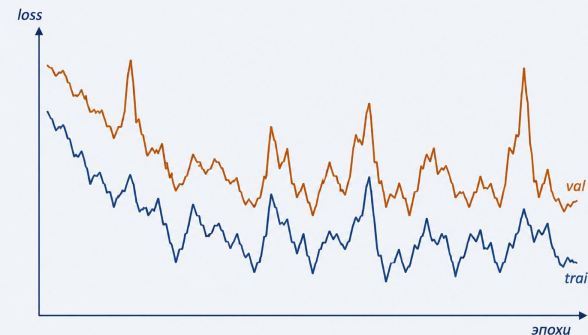
Кривые прыгают и не сходятся

Симптом: потери скачут от эпохи к эпохе, не выходят на устойчивое снижение, иногда внезапно растут. Это не недо- и не переобучение – это сломанный процесс обучения.

Частые причины и что делать:

- слишком большой learning rate – уменьшить;
- неудачная инициализация – поменять схему;
- немасштабированные входы – нормализовать признаки;
- batch normalization для стабилизации.

Нестабильное обучение



кривые сильно колеблются и не выходят на устойчивое снижение

ПРИЗНАК

«Зубцы» вместо гладкого спуска. Сначала чините процесс, потом судите о качестве.



Малая и шумная выборка – идеальные условия для самообмана

ИБ-КЕЙС · КЛАССИФИКАЦИЯ SOC-АЛЕРТОВ

Задача: отделить истинные инциденты от ложных срабатываний по ~3000 размеченных алертов SOC – выборка маленькая и шумная (метки ставили разные аналитики). **Модель:** MLP с тремя скрытыми слоями, без регуляризации, обучение на 60 эпох.

Что показали кривые и метрики:

- train accuracy дошла до 0.99, train loss – почти ноль: сеть «выучила» обучающую выборку;
- validation loss минимален к ~12-й эпохе, затем неуклонно растёт – классические «ножницы»;
- на новых алертах следующей недели качество просело до уровня чуть лучше случайного.

ВЫВОД

На малых шумных ИБ-данных сложная сеть запоминает шум. Диагноз ставится по кривым, а не по итоговой train accuracy.



Ловушка уверенности softmax

45 / 65

Часть 3 · Блок D

Высокий score – это ещё не высокая вероятность правоты

Softmax всегда выдаёт число в $(0, 1)$, и его легко принять за вероятность правильности. Но обученная без калибровки сеть склонна быть **чрезмерно уверенной**: она может выдать score 0.98 на объекте, который видит впервые и классифицирует неверно.

ЧАСТАЯ ОШИБКА · SCORE = ВЕРОЯТНОСТЬ

«Сеть выдала 0.98 – значит, она права на 98%». Score отражает лишь относительный порядок классов внутри модели, а не реальную частоту правоты. Особенно ненадёжен на данных, отличных от обучающих.

ПОЧЕМУ ЭТО ОПАСНО В ИБ

Автоматическая блокировка по «уверенному» score 0.98 на сдвинувшемся трафике может массово блокировать легитимных пользователей или пропускать атаки. **Что делать:** калибровать вероятности (как в L07), выбирать порог под цену ошибки и держать human-in-the-loop там, где цена ошибки высока.



Что проверить, прежде чем сказать «модель обучилась»

Прежде чем доверять метрике и выводить модель дальше, пройдите по пунктам. Это занимает минуты и отсекает большинство тихих ошибок.

- 1 Построены обе кривые?** – train и validation loss на одних осях, а не одна итоговая accuracy.
- 2 Обе ли снижаются?** – если нет – недообучение или сломанный процесс (нестабильность).
- 3 Растёт ли val loss?** – разворот вверх = переобучение; зафиксировать early stopping.
- 4 Гладкий ли спуск?** – скачки – сигнал уменьшить learning rate и проверить вход.
- 5 Зафиксирован ли seed?** – иначе результат неповторим и сравнение моделей невозможно.
- 6 Не приняли ли score за вероятность?** – softmax-уверенность требует калибровки перед использованием.



Диагностика – главный практический навык этой лекции

Что мы только что разобрали:

- ✓ Качество модели определяет validation loss, а не train loss и не итоговая accuracy. Всегда смотрите обе кривые.
- ✓ Недообучение – обе потери высоки; переобучение – train вниз, val вверх («ножницы»); нестабильность – кривые скачут.
- ✓ На малых и шумных ИБ-данных (SOC-алерты) сложная сеть запоминает шум; диагноз ставится по кривым, а не по train accuracy.
- ✓ Высокий softmax-score ≠ надёжная вероятность: сеть бывает чрезмерно уверена, что опасно при автоматических решениях в ИБ.
- ✓ Минимальная диагностика (обе кривые, разрыв, разворот, гладкость, seed, калибровка) занимает минуты и отсекает тихие ошибки.



Когда MLP оправдан

Выбор между MLP, бустингом и линейным baseline

В этом блоке:

- прикладной критерий выбора модели;
- явная позиция курса: MLP vs бустинг на табличных данных;
- частая ошибка – MLP без сравнения с baseline;
- когда сеть действительно нужна.



Что реально определяет, нужна ли здесь нейросеть

Выбор модели – инженерное решение, а не вопрос моды. Перед тем как брать MLP, ответьте на четыре вопроса:

Тип данных

табличные – сильны деревья и бустинг; неструктурированные (изображения, текст, сырые последовательности) – преимущество за сетью.

Размер выборки

мало данных – сложная сеть переобучается; бустинг и линейные модели устойчивее на малых выборках.

Стоимость настройки

сеть требует подбора архитектуры, lr, регуляризации; бустинг даёт сильный результат почти без настройки.

Требование к объяснимости

там, где нужно объяснять решения (аудит, расследование) – проще интерпретируемые модели.



Явная позиция курса, а не личное предпочтение

ПОЗИЦИЯ КУРСА

На типичных табличных ИБ-задачах (NetFlow-признаки, статические признаки файлов, агрегаты транзакций) MLP, как правило, НЕ превосходит хорошо настроенный градиентный бустинг – и при этом требует заметно больше усилий по настройке.

Почему так:

- табличные признаки разнородны и часто слабо связаны – деревья работают с ними естественно;
- бустинг устойчив к масштабу признаков и пропускам, почти не требует подготовки;
- сети нужны большие однородные данные, чтобы раскрыть преимущество, – а таблицы ИБ обычно не такие.

ЛИНИЯ BASELINE-ДИСЦИПЛИНЫ

MLP должен сравниваться с бустингом (L08) и линейным baseline (L06), а не браться по умолчанию. Это продолжение дисциплины из L03.



Ошибка: MLP без сравнения

51 / 65

Часть 3 · Блок E

Архитектуру нельзя выбрать «на веру»

ЧАСТАЯ ОШИБКА · ПРОПУЩЕННЫЙ BASELINE

«Взял MLP, получил 0.97 – отличная модель».

Инженер применяет нейросеть к табличной задаче, не обучив для сравнения ни бустинг, ни линейный baseline. Цифра 0.97 сама по себе ничего не говорит: может, простое дерево давало 0.98 за минуту.

ПОЧЕМУ ЭТО НЕВЕРНО

Без baseline нельзя обосновать выбор архитектуры. На защите проекта или код-ревью вопрос «почему сеть, а не бустинг?» останется без ответа. Метрика без точки отсчёта неинтерпретируема, а более сложная и дорогая модель без доказанного выигрыша – плохое инженерное решение.

Правило: сначала линейный baseline и бустинг, потом – сеть, и только если она честно выигрывает с учётом стоимости.



Случаи, где нейросеть – правильный выбор

Сдержанная позиция к табличным задачам не означает «сети не нужны». Есть случаи, где сеть – именно то, что требуется:

- ✓ **Неструктурированные данные** – изображения, тексты, сырые байтовые или сетевые последовательности – там, где ручные признаки не работают.
- ✓ **Перенос представлений** – предобученные сети дают сильные эмбединги, которые трудно воспроизвести вручную.
- ✓ **База для специализации** – MLP как baseline и фундамент перед CNN (L14) и трансформерами для текста (L15).
- ✓ **Очень большие однородные данные** – когда объёма достаточно, чтобы сеть раскрыла гибкость без переобучения.



Дисциплина выбора модели вместо моды на нейросети

Что мы только что разобрали:

- ✓ Выбор модели определяют тип данных, размер выборки, стоимость настройки и требование к объяснимости – а не «мощность» архитектуры.
- ✓ Позиция курса: на табличных ИБ-задачах MLP, как правило, не превосходит бустинг и требует больше усилий.
- ✓ MLP всегда сравнивают с бустингом (L08) и линейным baseline (L06); без точки отсчёта метрика неинтерпретируема.
- ✓ Сеть оправдана на неструктурированных данных, для переноса представлений и как фундамент для CNN (L14) и текстов (L15).
- ✓ Более сложная и дорогая модель без доказанного выигрыша – плохое инженерное решение, а не достижение.

Честное сравнение: по качеству И по стоимости

Та же задача обнаружения атак в сетевом трафике, что решал бустинг в L08, – теперь с MLP на тех же признаках. Сравниваем не только метрику, но и затраты.

Критерий	Бустинг (L08)	MLP	Комментарий
ROC-AUC	0.985	0.983	сопоставимо, разница в пределах шума
PR-AUC	0.94	0.93	сопоставимо
Время обучения	~3 мин	~35 мин	MLP дороже на порядок
Настройка	минимальная	слои, lr, dropout, эпохи	бустинг почти из коробки
Объяснимость	feature importance	сложнее	бустинг проще для аудита

Вывод

Качество – то же, стоимость – в разы выше. На этой табличной задаче бустинг остаётся рациональным выбором.



Почему сети редко выигрывают на ИБ-таблицах

Табличные данные информационной безопасности обладают свойствами, которые играют против нейросетей и в пользу деревьев:

Разнородные признаки

счётчики, флаги, длительности, категории в одной таблице – деревья обрабатывают их естественно, сети требуют масштабирования.

Сильный дисбаланс классов

атак – доли процента; сеть легко скатывается в «всё норма», бустинг устойчивее с правильной функцией потерь.

Шум и артефакты разметки

метки от разных аналитиков, ошибки логирования – сложная сеть запоминает эти артефакты.

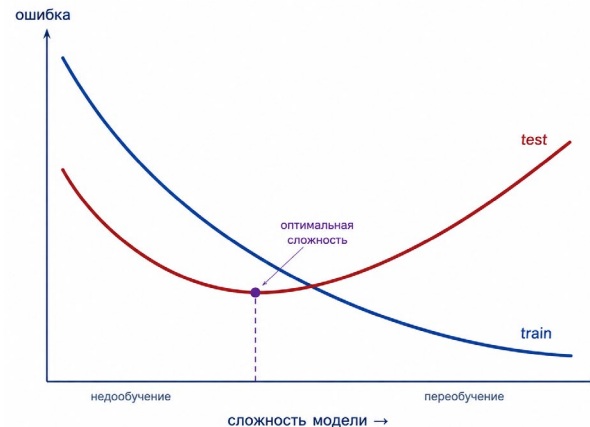
Умеренный объём

часто десятки-сотни тысяч строк, а не миллионы – сети не хватает данных, чтобы раскрыть гибкость.



Худшие условия для сложной модели

Чем меньше и шумнее данные, тем опаснее давать модели большую ёмкость. На малой выборке сеть с лёгкостью находит **ложные закономерности** – комбинации признаков, случайно совпавшие с меткой в обучении, но бесполезные на новых данных.



ПРАВИЛО

Мало данных → проще модель. Линейный baseline или неглубокий бустинг почти всегда надёжнее MLP.



Сеть может ломаться незаметно

Сетевой трафик и поведение атакующих меняются со временем: появляются новые сервисы, протоколы, типы атак. Это **сдвиг данных (data drift)**. Модель, обученная на старом распределении, на новом постепенно теряет качество – но метрики на старой валидации этого не показывают.

ОСОБЕННОСТЬ НЕЙРОСЕТЕЙ

Сеть с чрезмерной уверенностью продолжает выдавать высокие softmax-score даже на сместившихся данных, которые классифицирует неверно. Деградация не сопровождается «честным» падением уверенности – её легко не заметить.

Что с этим делать: мониторить распределение входов и качество на свежих данных, периодически переобучать, держать человека в контуре. Подробно про устойчивость к сдвигу – в L17.

Сдвиг данных как отдельная тема (covariate shift, concept drift, OOD) – лекция L17.



Когда виноваты не данные, а настройка процесса

ИБ-КЕЙС · ДЕТЕКТОР АНОМАЛИЙ В ЛОГАХ

Симптом: MLP на признаках логов аутентификации не сходился – loss скакал, разные запуски давали то 0.6, то 0.9 ROC-AUC. Команда заподозрила «плохие данные». **Реальная причина:** немасштабированные входы (счётчики до десятков тысяч рядом с флагами 0/1) и слишком большой learning rate.

Что помогло:

- нормализация входных признаков к сопоставимому масштабу;
- уменьшение learning rate и фиксация seed для воспроизводимости;
- batch normalization – после этого кривая стала гладкой и стабильной от запуска к запуску.

Урок: «плохой результат» сети – часто диагноз процессу обучения, а не данным. Сначала читаем кривую.



Короткий мост к теме состязательных воздействий

Нейросети особенно уязвимы к **состязательным возмущениям (adversarial perturbations)**: малозаметное, специально подобранное изменение входа способно заставить уверенную сеть ошибиться. Гладкость и высокая размерность решающей функции, которые дают сети гибкость, одновременно открывают такие лазейки.

ПОЧЕМУ ЭТО ВАЖНО В ИБ

В ИБ у модели есть мотивированный противник, которому выгодно её обмануть: обойти детектор вредоносного трафика, спам-фильтр или антивирус. Поэтому уязвимость сетей – не абстракция, а часть модели угроз ML-компонента.

Это только мост. Состязательные атаки (evasion, FGSM, PGD, transferability) и меры защиты – отдельная большая тема лекции L18. Здесь важно лишь запомнить: гибкость сети имеет обратную сторону в виде уязвимости.



Сводка специфичных капканов этой темы



Сеть «по умолчанию» на таблице – не сравнили с бустингом и линейным baseline – выбор архитектуры необоснован.



Оценка по одной ассигасу – на дисбалансе атак ассигасу обманчива; нужны PR-AUC, ROC-AUC и обе кривые потерь.



Доверие softmax-score – высокую уверенность приняли за вероятность; для решений нужна калибровка.



Сложная модель на малых данных – сеть запоминает шум SOC-разметки; проще модель надёжнее.



Игнор сдвига данных – модель тихо деградирует на новом трафике без мониторинга.



Семь ключевых утверждений

- 1 Нейросеть – не «чёрный ящик-самоучка»: качество зависит от данных, потерь, оптимизации, регуляризации и диагностики.
- 2 MLP – это стопка полносвязных слоёв с нелинейностями; обучение – спуск по поверхности потерь по антиградиенту (backprop).
- 3 Качество определяет validation loss, а не train loss и не итоговая accuracy. Всегда смотрите обе кривые.
- 4 Три режима по кривым: недообучение (обе высоко), переобучение («ножницы»), нестабильность (скачки).
- 5 Регуляризация (dropout, weight decay, early stopping) и фиксация seed – обязательная гигиена обучения.
- 6 Softmax-score $\neq$ надёжная вероятность; на сместившихся данных сеть деградирует тихо. В ИБ это особенно опасно.
- 7 На табличных ИБ-задачах MLP сравнивают с бустингом и baseline, а не берут по умолчанию.



Карта применимости и навигация по курсу

ПРИМЕНЯТЬ СЕТЬ

- неструктурированные данные (изображения, текст, сырые последовательности);
- перенос предобученных представлений;
- очень большие однородные выборки.

ОСТАТЬСЯ НА БУСТИНГЕ

- табличные ИБ-признаки (NetFlow, файлы, транзакции);
- малые и шумные выборки;
- когда важна объяснимость и низкая стоимость настройки.

Что читать дальше:

- L14** CNN для изображений и переносимость качества
- L15** Тексты и логи: от TF-IDF к эмбедингам и трансформерам
- L17** Устойчивость к шуму и сдвигу данных
- L18** Состязательные воздействия на модели
- L20** Запоминание данных и приватность



Главы учебников и специализированная литература

Обязательное:

- Bishop, Pattern Recognition and Machine Learning – гл. 5 (Neural Networks).
- Hastie, Tibshirani, Friedman, The Elements of Statistical Learning – гл. 11 (Neural Networks) и гл. 7 (Model Assessment and Selection).
- Goodfellow, Bengio, Courville, Deep Learning – гл. 6 (Deep Feedforward Networks), гл. 7 (Regularization), гл. 8 (Optimization).

Дополнительно и практика:

- документация scikit-learn – MLPClassifier (Neural network models, раздел Supervised learning);
- PyTorch – официальный туториал «Learn the Basics» (tensors, autograd, training loop);
- разбор learning curves и диагностики обучения в документации scikit-learn (Validation curves).



С чем работать в проекте

СКВОЗНОЙ ДАТАСЕТ · CIC-IDS-2018

Публичный датасет сетевого трафика (Canadian Institute for Cybersecurity). ≈ 16 млн flow-записей, 80 признаков, 14 типов атак + Benign. Используйте те же признаки, что и для бустинга в L08, чтобы сравнение MLP и бустинга было честным.

Инструменты:

- scikit-learn – MLPClassifier для быстрого baseline и честного сравнения с бустингом;
- PyTorch – для полного контроля над архитектурой, обучением и логированием кривых;
- matplotlib / TensorBoard – для построения и чтения learning curves.

Совет: всегда логируйте train и validation loss по эпохам – без них диагностика невозможна.



Ключевые термины лекции

MLP	многослойный персептрон — сеть из полносвязных слоёв	Learning rate	размер шага градиентного спуска
Forward pass	прямой проход: вычисление ответа сети по слоям	Dropout	случайное отключение нейронов при обучении
Backpropagation	обратный проход: вычисление градиента через chain rule	Weight decay	L2-штраф за большие веса (регуляризация)
Loss / $L(w)$	функция потерь; для классификации — кросс-энтропия	Early stopping	остановка в минимуме validation loss
Optimizer	алгоритм обновления весов (SGD, Adam)	Learning curves	графики train и validation loss по эпохам
Epoch	один полный проход по обучающим данным	Переобучение	train падает, validation растёт («ножницы»)
Batch	мини-выборка для одного шага обновления	Softmax	перевод логитов в распределение по классам